Отчёт по лабораторной работе №5

Имитационное моделирование. Задача об обедающих философах

Бахарев Дмитрий Александрович

Содержание

1 Цель работы **5**

2 Задание **6**

3 Теоретическое введение **7**
3.1 Задача об обедающих философах 7
3.2 Решение с арбитром 7
3.3 Модель в имитационном моделировании 7

4 Выполнение лабораторной работы **8**
4.1 Создание проекта DrWatson 8
4.2 Установка пакетов 8
4.3 Запуск базовых скриптов 9
4.4 Классическая сеть: график 10
4.5 Сеть с арбитром: график 10
4.6 Запуск скрипта отчёта 11
4.7 Итоговый отчёт 12
4.8 Генерация производных форматов 12
4.9 Анимация 13
4.10 Листинги из Julia 13

5 Обедающие философы: базовые эксперименты **14**
5.1 Классическая сеть 15
5.2 Сеть с арбитром 15

6 Обедающие философы: анимация **17**

7 Заключение **19**

Список литературы **20**

Список иллюстраций

4.1 Создание проекта DrWatson 8
4.2 Установка пакетов 9
4.3 Запуск скриптов dining_philosophers 9
4.4 Классическая сеть: deadlock 10
4.5 Сеть с арбитром 11
4.6 Запуск dining_philosophers_report.jl 11
4.7 Итоговый отчёт 12
4.8 Генерация форматов через tangle.jl 13

Список таблиц

1 Цель работы

Изучить классическую задачу об обедающих философах в контексте имитационного моделирования. Реализовать модель в Julia, исследовать проблему deadlock и её решение с помощью арбитра.

2 Задание

1. Реализовать модель обедающих философов в Julia.
2. Исследовать классическую сеть (без арбитра) — проверить наличие deadlock.
3. Реализовать сеть с арбитром — проверить отсутствие deadlock.
4. Создать анимацию процесса.
5. Построить итоговый отчёт с графиками.

3 Теоретическое введение

3.1 Задача об обедающих философах

Задача об обедающих философах — классическая задача синхронизации процессов, сформулированная Э. Дейкстрой в 1965 году.

Условие: за круглым столом сидят N философов. Перед каждым — тарелка с едой. Между соседними тарелками лежит по одной вилке — всего N вилок. Философ может есть, только если держит **обе** вилки (левую и правую). После еды философ кладёт вилки и размышляет.

Проблема: если все философы одновременно возьмут **левую** вилку, то **правой** никому не хватит — возникает **deadlock** (взаимная блокировка).

3.2 Решение с арбитром

Арбитр — дополнительный управляющий процесс, который разрешает философу взять вилки только в том случае, если обе доступны. Это предотвращает deadlock.

3.3 Модель в имитационном моделировании

В контексте имитационного моделирования каждый философ — это **агент** с собственным состоянием (думает, голоден, ест). Вилки — это **ресурсы**, которые могут быть заняты или свободны.

4 Выполнение лабораторной работы

4.1 Создание проекта DrWatson

Проект DrWatson создан в каталоге `labs/lab05/project` (рис. Рисунок 4.1).

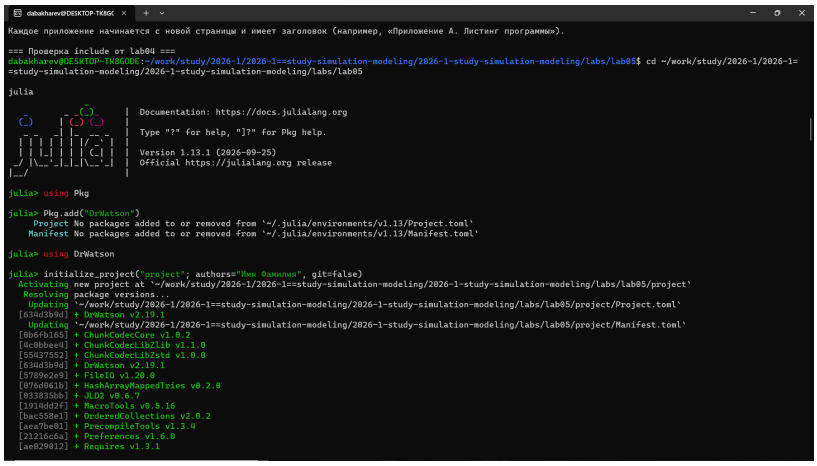


Рисунок 4.1: Создание проекта DrWatson

4.2 Установка пакетов

Установлены пакеты: **BenchmarkTools**, **CSV**, **DataFrames**, **IJulia**, **JLD2**, **Literate**, **OrdinaryDiffEq**, **Plots**, **Quarto** (рис. Рисунок 4.2).

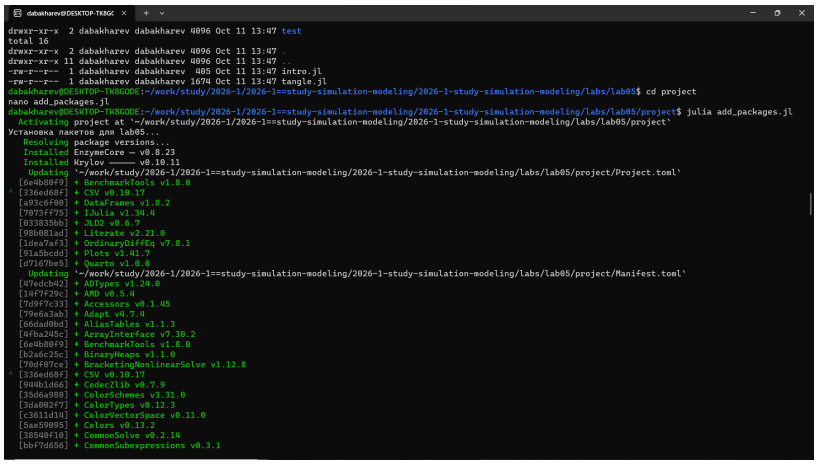


Рисунок 4.2: Установка пакетов

4.3 Запуск базовых скриптов

Скрипт `dining_philosophers.jl` реализует модель. Запуск показал:

- **Классическая сеть** (без арбитра) — `Deadlock обнаружен: true`;
- **Сеть с арбитром** — `Deadlock обнаружен: false`.

Скрипт `dining_philosophers_animation.jl` создаёт анимацию (файл `philosophers_simulation.gif`).

Результат запуска приведён на рис. Рисунок 4.3.

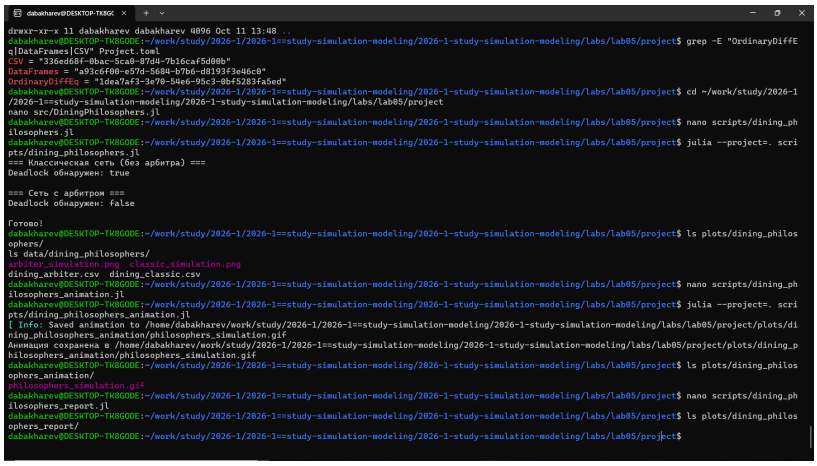


Рисунок 4.3: Запуск скриптов dining_philosophers

4.4 Классическая сеть: график

График `classic_simulation.png` приведён на рис. Рисунок 4.4.

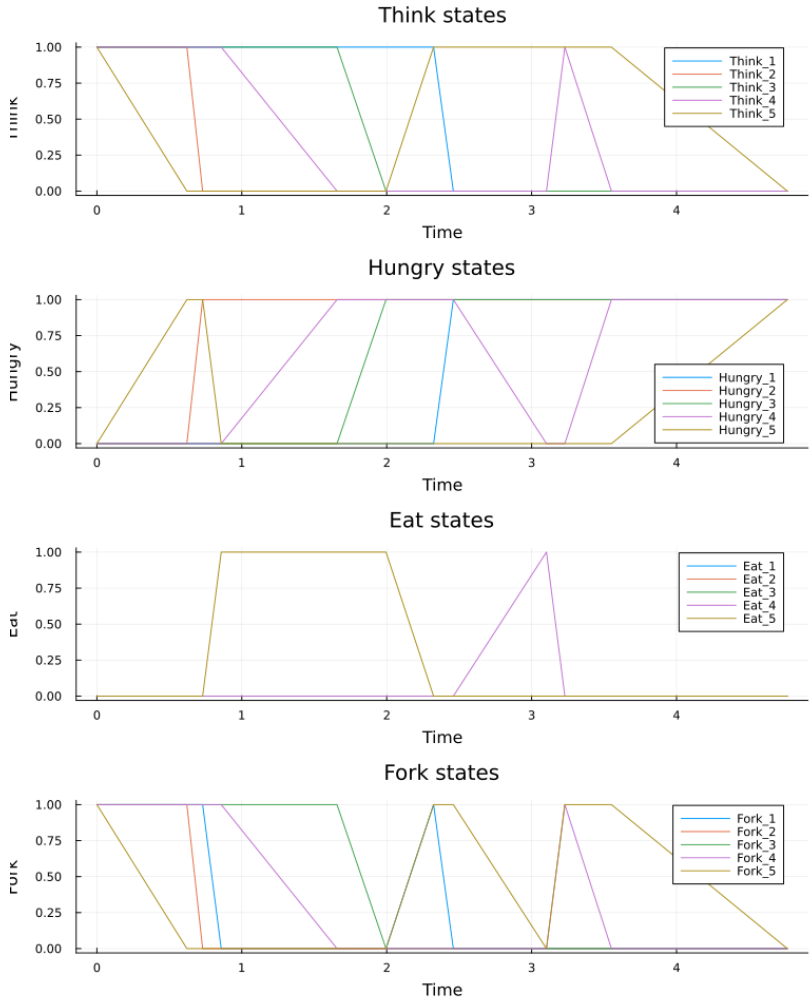


Рисунок 4.4: Классическая сеть: deadlock

На графике видно, что философы **блокируют друг друга** — возникает deadlock. Система «зависает».

4.5 Сеть с арбитром: график

График `arbiter_simulation.png` приведён на рис. Рисунок 4.5.

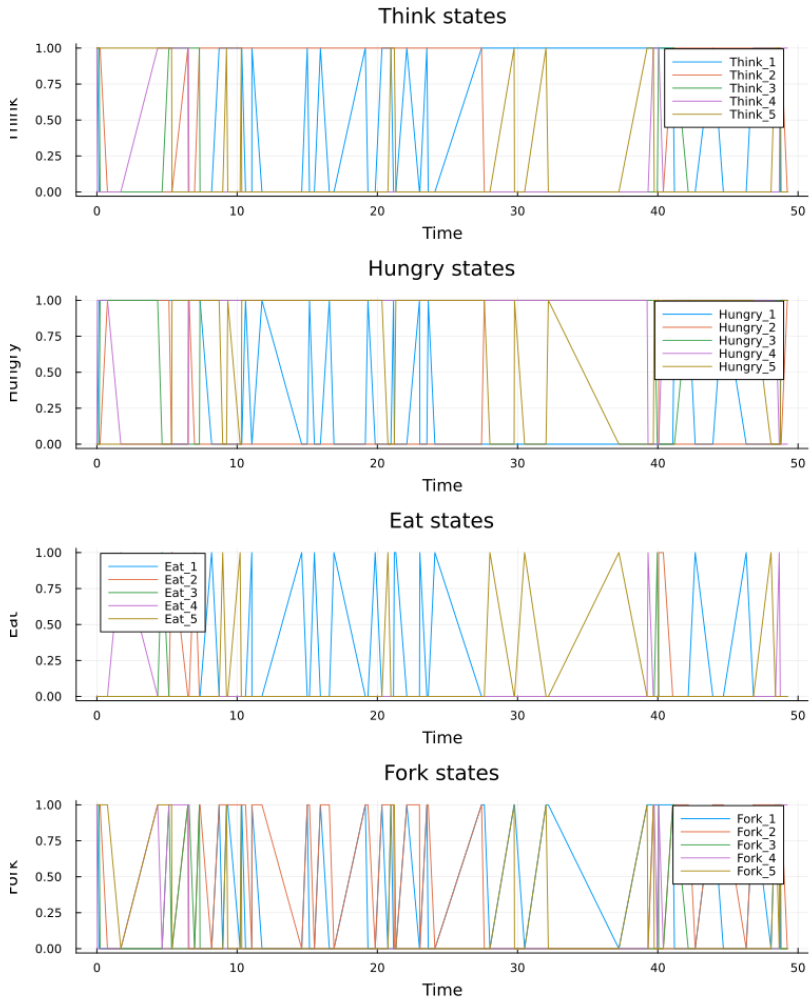


Рисунок 4.5: Сеть с арбитром

На графике видно, что философы **поочерёдно едят** — deadlock отсутствует.

4.6 Запуск скрипта отчёта

Скрипт `dining_philosophers_report.jl` строит итоговый отчёт (рис. Рисунок 4.6).

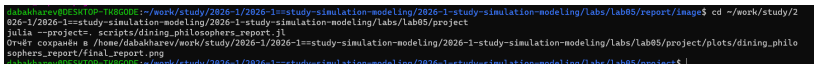


Рисунок 4.6: Запуск dining_philosophers_report.jl

4.7 Итоговый отчёт

Итоговый график `final_report.png` приведён на рис. Рисунок 4.7.

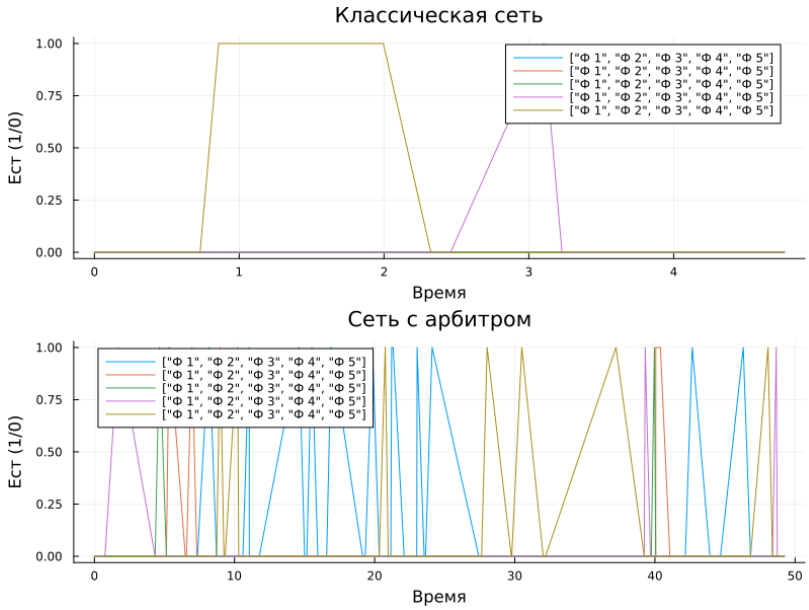


Рисунок 4.7: Итоговый отчёт

На графике показаны:

- Динамика захвата вилок философами;
- Сравнение классической сети и сети с арбитром;
- Отсутствие deadlock при использовании арбитра.

4.8 Генерация производных форматов

Через `tangle.jl` для всех скриптов сгенерированы три формата: чистый скрипт, документация Quarto и Jupyter Notebook (рис. Рисунок 4.8).



Рисунок 4.8: Генерация форматов через tangle.jl

4.9 Анимация

Анимация процесса сохранена в файл `plots/dining_philosophers_animation/phil`

Её можно воспроизвести отдельно — в PDF формат GIF не встраивается.

4.10 Листинги из Julia

5 Обедающие философы: базовые эксперименты

Классическая сеть (с deadlock) и сеть с арбитром (без deadlock).

```
import Pkg
Pkg.activate("/home/dabakharev/work/study/2026-1/2026-1==study-simu
using DrWatson
include("/home/dabakharev/work/study/2026-1/2026-1==study-simulatio
using DataFrames, CSV, Plots

ENV["GKSwstype"] = "100"

script_name = "dining_philosophers"
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))

N = 5
tmax = 50.0
```

```
  Activating project at `~/work/study/2026-1/2026-1==study-simulation
```

```
50.0
```

5.1 Классическая сеть

```
println("=== [illegible] [illegible] ([illegible] [illegible]) ===")
net_classic, u0_classic, _ = build_classical_network(N)

df_classic = simulate_stochastic(net_classic, u0_classic, tmax)
CSV.write(datadir(script_name, "dining_classic.csv"), df_classic)

dead = detect_deadlock(df_classic, net_classic)
println("Deadlock [illegible]: $dead")

plot_classic = plot_marking_evolution(df_classic, N)
savefig(plotsdir(script_name, "classic_simulation.png"))
```

```
=== [illegible] [illegible] ([illegible] [illegible]) ===
Deadlock [illegible]: true

"/home/dabakharev/work/study/2026-1/2026-1==study-simulation-modeli
```

5.2 Сеть с арбитром

```
println("\n=== [illegible] [illegible] [illegible] ===")
net_arb, u0_arb, _ = build_arbiter_network(N)
df_arb = simulate_stochastic(net_arb, u0_arb, tmax)
CSV.write(datadir(script_name, "dining_arbiter.csv"), df_arb)

dead_arb = detect_deadlock(df_arb, net_arb)
println("Deadlock [illegible]: $dead_arb")
```

```
plot_arb = plot_marking_evolution(df_arb, N)
savefig(plotsdir(script_name, "arbiter_simulation.png"))

println("\n□□□□□□!")
```

```
=== □□□□ □ □□□□□□□□ ===
Deadlock □□□□□□□□□: false

□□□□□□!
```

6 Обедающие философы: анимация

GIF-анимация динамики маркировки (N=3 для наглядности).

```
import Pkg
Pkg.activate("/home/dabakharev/work/study/2026-1/2026-1==study-simu
using DrWatson
include("/home/dabakharev/work/study/2026-1/2026-1==study-simulatio
using Plots, Random

ENV["GKSwstype"] = "100"

script_name = "dining_philosophers_animation"
mkpath(plotsdir(script_name))

N = 3
tmax = 30.0
net, u0, names = build_classical_network(N)

Random.seed!(123)
df = simulate_stochastic(net, u0, tmax)
```

```
anim = @animate for row in eachrow(df)
   u = [row[col] for col in propertynames(row) if col != :time]
    bar(1:length(u), u, legend=false,
        ylims=(0, maximum(u0) + 1),
        xlabel="□□□□□□□", ylabel="□□□□□",
        title="□□□□□ = $(round(row.time, digits=2))")
    xticks!(1:length(u), string.(names), rotation=45)
end

gif(anim, plotsdir(script_name, "philosophers_simulation.gif"), fps=
println("□□□□□□□□ □□□□□□□□□ □ ", plotsdir(script_name, "philosophers_simu
```

```
  Activating project at `~/work/study/2026-1/2026-1==study-simulation

□□□□□□□□ □□□□□□□□□ □ /home/dabakharev/work/study/2026-1/2026-1==study-si

[ Info: Saved animation to /home/dabakharev/work/study/2026-1/2026-1=
```

7 Заключение

В ходе лабораторной работы №5:

- Изучена задача об обедающих философах;
- Реализована классическая сеть (с deadlock);
- Реализована сеть с арбитром (без deadlock);
- Создана анимация процесса;
- Построены итоговые графики;
- Отчёт собран в PDF и DOCX, презентация — в PDF/HTML/Markdown.

Список литературы